

Quick Start Guide

This guide covers scene setup, car logic, and basic visuals — by the end, you will have a fully functional car with visuals, ready to drive.

Note

All components are self-documented in the inspector. Hover over any field for a tooltip.

Note

A pre-configured `QuickStartScene` is provided at `LuigiGameDev/CarController/Sample`. Use it as a reference alongside this guide.

Scene Setup

By the end of this section, you will have a scene with all necessary systems in place, ready for a car to be added and driven.

Step 1: Create New Scene

- Create a new scene.
- Add a Plane as the ground and reset its Transform to the origin (0, 0, 0).
- Set the Plane scale to (20, 20, 20).

Step 2: Create Player Controller

Handles player input and forwards it to the car.

- Create a new GameObject and attach the `PlayerController` script (not Unity's built-in).
- Assign the `InputActionAsset` reference. A sample asset is provided at `LuigiGameDev/CarController/Sample/Input`.

Step 3: Setup Camera

Cinemachine is used for smooth camera follow behavior.

- Create a new Cinemachine Camera: Right-click the Hierarchy → Cinemachine → Cinemachine Camera.
- Set **Position Control** to `Follow`.
- Set **Rotation Control** to `Rotation Composer`.
- Set **Binding Mode** to `Lock To Target With World Up`.

Step 4: Create Scene Script

Assembles all systems and integrates the car controller for the scene. `SampleSceneScript` is provided as a working example and starting point, not a core package component.

- Create a new GameObject and attach the `SampleSceneScript` (`LuigiGameDev.CarController.Sample`).
 - Assign the `InputActionAsset` reference. A sample asset is provided at `LuigiGameDev/CarController/Sample/Input`.
 - Assign the `PlayerController` reference.
 - Assign the `CinemachineCamera` reference.
-

Car Logic Setup

By the end of this section, you will have a fully functional car ready to drive. Visual elements are not included yet — use gizmos for visualization and debugging.

Step 1: Create the Car GameObject

- Create an empty GameObject and attach the `Car` script.
 - Automatically adds: `BoxCollider`, `Rigidbody`, `CarSettings`, `CarInput`.
 - `CarSettings`: exposes all car physics parameters for configuration. (Hover any field for tooltips.)
 - `CarInput`: input interface the `PlayerController` uses to communicate with the car.
- Set the car Position Y to `1` — the car will fall to the ground on play, activating the springs.
- Set Rotation to `(0, 0, 0)` and Scale to `(1, 1, 1)`.

Step 2: Setup Collider and Rigidbody

- Set **BoxCollider Size** to `(1.7, 0.4, 4)`.
- Set **Rigidbody Mass** to `100`.
- Set **Rigidbody Interpolate** to `Interpolate` — ensures smooth visual movement between physics steps.

Step 3: Generate Visualization Mesh

Generates a mesh matching the BoxCollider for visualization in the absence of car visuals.

- Right-click the `Car` component in the inspector → `GenerateCollidersMesh`.

Step 4: Test It

- In the `Car` inspector, enable **Gizmos Draw Wheel Transform** and **Gizmos Draw Spring**.
- Enable gizmo rendering in the Scene view — click the **Gizmos** button in the toolbar.
- Enable gizmo rendering in the Game view — click the **Gizmos** button in the Game view toolbar.
- Hit play and drive.



Tip

A joystick or gamepad is recommended for the best experience.

Car Visuals Setup

By the end of this section, you will have integrated basic visuals to the functional car.

Step 1: Create View State Linker

Bridges car logic state to the view layer for visuals.

- Create a new GameObject and attach the `ViewStateLinker` script.
- Assign the `ViewStateLinker` reference in the `SampleSceneScript` inspector.

Step 2: Add Car Mesh

- Add `CarMesh.prefab` as a child of the car GameObject. Found at `LuigiGameDev/CarController/Sample/Prefabs/Vehicles`.
- Set its relative Position to `(0, -0.6, 0)`, Rotation to `(0, 0, 0)`, and Scale to `(1, 1, 1)`.

Step 3: Add Car View State

Holds car state for view scripts — visuals, audio, and effects. Populated at runtime by the `ViewStateLinker`.

- Attach the `CarViewState` component to the car `GameObject`.

Note

Inspector fields are exposed for debugging and manual testing of view scripts.

Step 4: Add Car Wheels Visual

Animates wheel position, spin, and steering visuals based on car state.

- Attach the `CarWheelsVisual` component to the car `GameObject`.
- Assign the **Wheel Transforms** references in order: FL, FR, RR, RL.
- Tune `FullyCompressedY` and `FullyExtendedY` to match your car mesh.
 - These define the wheel Y offset at full compression and full extension.
 - Use the **Spring Extensions** sliders in the inspector to preview and fine-tune without entering play mode — drag the slider to simulate suspension travel.
 - `0` = fully compressed, `1` = fully extended.
 - Default values work for the sample `CarMesh.prefab`.

Note

`CarWheelsVisual` reads car state from `CarViewState`, not directly from the car logic. This is the core pattern of the view architecture — view scripts are decoupled from logic and communicate only through `CarViewState`.

Step 5: Test It

- Hit play and drive.

Tip

Use left/right on the keyboard or gamepad to switch between cars.
